Table des matières

Cours 10 — Passer à un format de table ouvert 1
Objectifs 2
1. Ce qu'un dossier de Parquet ne sait pas faire 3
2. Une table, ce sont des fichiers et des métadonnées 3
3. Le catalogue, et une validation qui tient en un pointeur 5
4. L'évolution du schéma : par identifiant, pas par nom 6
5. Le partitionnement caché 7
6. Les instantanés et le voyage dans le temps 7
7. L'entretien : compacter, expirer, nettoyer 8
8. Iceberg et Delta : deux formats, la même idée 9
9. Le médaillon : bronze, argent, or 10
10. Le médaillon et la chaîne `RAW`, `STAGING`, `MARTS` 11
11. Le lakehouse, et la convergence du lac et de l'entrepôt 12
Points clés à retenir 13
Exercices 14

Cours 10 — Passer à un format de table ouvert

Théorie · Cours 10/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → ingestion → **lac** → entrepôt → restitution. Depuis le Cours 02, le lac de Boréal est un dossier de fichiers Parquet, un par jour livré. Ce cours y pose des tables : les mêmes fichiers, plus des métadonnées qui disent lesquels forment la table, à quel moment, et sous quel schéma. C'est ce qui rapproche le lac de l'entrepôt, jusqu'à ce qu'on appelle un lakehouse.

En 30 secondes :

- Un dossier de Parquet ne sait ni valider une écriture d'un seul coup, ni renommer une colonne, ni dire ce qu'il contenait hier. Le bronze de la télémétrie porte deux colonnes pour la même température depuis le micrologiciel 2.0, et chaque lecteur doit s'en souvenir.

© 2026 iAlexMG. Tous droits réservés.

- Une table Iceberg, ce sont des fichiers Parquet, plus un arbre de métadonnées ; une table Delta, des fichiers Parquet, plus un journal. Dans les deux cas, valider une écriture revient à publier un nouvel état d'un seul geste.
- Iceberg désigne chaque colonne par un identifiant. Renommer la température ne réécrit aucun des 28 fichiers : ceux de 2023 se relisent sous le nom neuf.
- Chaque validation laisse un instantané. L'étiquette `fin-2025` rend l'état au 31 décembre, message pour message, même après l'entretien. Le voyage dans le temps suit pourtant l'heure des validations, pas celle des données.
- Une table ouverte se paie en entretien : compacter les petits fichiers, expirer les vieux instantanés, effacer ce que plus rien ne cite. Sans cela, ses métadonnées finissent par peser plus lourd que ses données.
- Le médaillon range le lac en bronze, argent et or. Il recoupe `RAW`, `STAGING` et `MARTS` sans les recouvrir, et le parcours dit qui tient quoi.

L'idée du cours. Un format de fichier dit comment ranger des lignes dans un fichier. Un format de table dit quels fichiers forment la table. C'est une question de plus, et tout ce qu'une base de données sait faire en dépend : valider d'un coup, changer de schéma, revenir en arrière. Le lac gagne ces trois gestes sans rien perdre de ce qui le rendait intéressant, des fichiers ouverts que n'importe quel moteur lit.

Objectifs

- Dire ce qu'un dossier de fichiers ne sait pas faire, et ce qu'un format de table y ajoute.
- Décrire l'arbre de métadonnées d'Iceberg et le journal de Delta, et lire l'un et l'autre.
- Expliquer une validation atomique par le remplacement d'un pointeur, et le rôle du catalogue.
- Faire évoluer un schéma sans réécrire de fichier, et dire pourquoi l'identifiant d'une colonne compte plus que son nom.
- Partitionner une table par une transformation cachée, et en voir l'effet sur ce qu'une requête lit.
- Remonter à un état passé par un instantané ou une étiquette, et distinguer l'heure des données de celle des validations.
- Tenir une table : compacter, expirer, nettoyer, et borner ses métadonnées.
- Comparer Iceberg et Delta Lake, et dire où ils convergent.
- Ranger un lac en bronze, argent et or, et situer ces couches face à `RAW`, `STAGING` et `MARTS`.
- Situer le lakehouse entre le lac et l'entrepôt, et reconnaître les offres clés en main.

© 2026 iAlexMG. Tous droits réservés.

1. Ce qu'un dossier de Parquet ne sait pas faire

Labo : `Labo_10_DE.md`, étape 1.

Le bronze de la télémétrie tient en 933 fichiers Parquet, un par jour de réception, rangés à la façon de Hive (`jour=2025-10-01/`). DuckDB les lit comme une seule table, et c'est une illusion commode : la table n'existe que dans la requête qui liste le dossier. Quatre choses lui manquent.

Ce qui manque	Ce qui se passe dans un dossier	Chez Boréal
Valider d'un coup	un lecteur qui liste le dossier pendant qu'on écrit voit l'écriture à moitié faite ; deux écrivains s'écrasent sans le savoir	le bronze se réécrit par jour livré, avec `OVERWRITE`
Changer de schéma	renommer une colonne veut dire réécrire tous les fichiers, ou vivre avec deux noms	`temp_moteur_max` et `temperature_moteur_max`, côte à côte depuis le 2025-05-01
Se souvenir	un fichier réécrit remplace l'ancien sans trace ; rien ne dit ce que le dossier contenait hier	l'état du 31 décembre ne se retrouve que si personne n'a rien touché depuis
Se décrire	le schéma, la partition et les statistiques se découvrent en ouvrant les fichiers	933 pieds de page à lire avant la première ligne

Le deuxième point se mesure. Le micrologiciel 2.0 a renommé la température du moteur et ajouté les heures du moteur. Le bronze déclare les deux noms depuis le Cours 02 : 88 678 messages remplissent l'ancien, 115 807 le nouveau, et chaque lecteur doit écrire `COALESCE` pour toujours. La bascule n'a pas été franche non plus : 9 835 messages reçus après le 1er mai 2025 portent encore l'ancien nom, publiés par des véhicules restés en 1.3 ou en 1.6 jusqu'à la mi-juillet.

Un format de table ouvert règle les quatre d'un même mouvement. Il ajoute aux fichiers de données des métadonnées, écrites elles aussi en fichiers, qui disent quels fichiers forment la table à un instant donné, sous quel schéma et quelle partition. Ouvert veut dire que la spécification est publique et que plusieurs moteurs la lisent et l'écrivent : DuckDB, Spark, Snowflake, Databricks, PyIceberg. Deux formats dominent, Apache Iceberg et Delta Lake ; un troisième, Apache Hudi, vise surtout les mises à jour fréquentes, et ce cours ne le traite pas.

2. Une table, ce sont des fichiers et des métadonnées

Labo : `Labo_10_DE.md`, étapes 3 et 9.

Iceberg range ses métadonnées en arbre. Une lecture part du haut et descend :

catalogue → fichier de métadonnées (`.metadata.json`) → liste de manifestes (`snap-….avro`) → manifestes (`….avro`) → fichiers de données (`.parquet`)

© 2026 iAlexMG. Tous droits réservés.

Niveau	Ce qu'il porte	Écrit quand
Catalogue	pour chaque table, le chemin de son fichier de métadonnées courant	à chaque validation, qui change ce chemin
Fichier de métadonnées	le schéma et ses versions, la spécification de partition, la liste des instantanés, les étiquettes	à chaque validation, un fichier neuf
Liste de manifestes	pour un instantané, les manifestes qui le composent, avec la plage des partitions de chacun	une par instantané
Manifeste	une liste de fichiers de données, avec pour chacun sa partition, son nombre de lignes et les bornes de chaque colonne	à chaque ajout
Fichier de données	les lignes, en Parquet	à chaque ajout

Au labo, 28 ajouts mensuels laissent 28 fichiers de données, 28 manifestes, 28 listes de manifestes et 31 fichiers de métadonnées : un par ajout, plus la création de la table, la pose de sa partition et l'évolution de son schéma. Les données pèsent 7,02 Mo ; les métadonnées, 0,79 Mo en tout. Rien ne s'efface : le fichier `00000` décrit toujours la table vide du premier instant.

Chaque niveau sert à lire moins. La liste de manifestes dit quelles partitions chaque manifeste couvre, et un lecteur saute les manifestes hors de son filtre ; le manifeste donne les bornes de chaque colonne de chaque fichier, et le lecteur saute les fichiers dont les bornes excluent sa requête. Personne ne liste de dossier : sur un stockage objet, où lister coûte une requête par millier de noms, c'est une économie qui compte.

Delta Lake range les siennes en journal. À côté des fichiers de données, le dossier `_delta_log/` tient un fichier JSON par version, numéroté de `00000000000000000000.json` en montant. Chaque fichier est une liste d'actions : `add` pour un fichier de données qui entre dans la table, avec ses statistiques ; `remove` pour un fichier qui en sort ; `metaData` quand le schéma change ; `commitInfo` pour dire qui a validé quoi. L'état de la table est la somme des actions depuis la version 0. Pour qu'un lecteur n'ait pas à relire des milliers de fichiers JSON, le protocole prévoit des points de reprise (*checkpoints*), des fichiers Parquet qui résument le journal jusqu'à une version. Au labo, les 31 versions de la télémétrie tiennent en 31 fichiers JSON ; celle de mai 2025 porte trois actions, puisqu'elle ajoute un fichier et change le schéma.

	Iceberg	Delta Lake
Forme des métadonnées	un arbre : métadonnées, listes de manifestes, manifestes	un journal : une suite de fichiers JSON, plus des points de reprise
Ce qu'une validation écrit	un fichier de métadonnées, une liste, un manifeste	un fichier JSON
Où est l'état courant	le fichier que le catalogue désigne	la dernière version du journal

© 2026 iAlexMG. Tous droits réservés.

	Iceberg	Delta Lake
Chemins des fichiers	absolus : déplacer la table casse ses métadonnées	relatifs au dossier de la table

La dernière ligne a une conséquence pratique. Les métadonnées d'Iceberg citent leurs fichiers par leur chemin complet, `C:/…/Projet_Boreal_intermediaire/plateforme/lac/iceberg/…` sur votre poste : copier le dossier ailleurs donne une table qui pointe encore vers l'ancien emplacement. On déplace une table Iceberg en la réécrivant, par l'outil qui l'a écrite.

3. Le catalogue, et une validation qui tient en un pointeur

Labo : `Labo_10_DE.md`, étapes 2 et 4.

Une base de données promet quatre propriétés à chaque transaction, qu'on résume par l'acronyme ACID : atomicité, tout ou rien ; cohérence, les règles de la table tiennent avant et après ; isolation, une transaction ne voit pas le travail inachevé d'une autre ; durabilité, ce qui est validé le reste. Un format de table les obtient sur des fichiers par un seul geste atomique, le remplacement d'un pointeur.

L'écrivain écrit d'abord tout ce qui est neuf : les fichiers de données, les manifestes, un nouveau fichier de métadonnées. Rien de tout cela n'est visible, puisque personne ne le désigne. Puis il demande au catalogue de remplacer le chemin courant par le sien, à une condition : que le chemin courant soit encore celui dont il est parti. Si oui, la table a changé d'état d'un seul coup ; sinon, un autre écrivain est passé entre-temps, et la validation est refusée.

Propriété	Ce qui la donne
Atomicité	un seul pointeur change ; les fichiers écrits avant sont invisibles tant qu'il ne change pas
Cohérence	le fichier de métadonnées porte le schéma, et l'écrivain vérifie ses données contre lui
Isolation	un lecteur suit un fichier de métadonnées fixe, et ne voit rien de ce qui se valide après
Durabilité	tout est écrit en fichiers avant que le pointeur bouge

L'isolation s'est vue au labo. La variable `telemetrie` de DuckDB tient un chemin de métadonnées ; après les 90 ajouts de 2026, elle désignait encore le fichier `00030`, et une requête sur elle aurait lu la table d'avant, entière et cohérente. Il a fallu relire `lac_iceberg.sql` pour suivre le pointeur.

La condition du remplacement s'appelle la concurrence optimiste : on ne verrouille rien, on vérifie au moment de valider. Deux écrivains lancés sur la même table, à l'écriture de ce cours, l'ont montré. Le second part d'un état que le premier a déjà remplacé ; PyIceberg 0.12 reçoit le refus, écrit `Commit failed due to a concurrent update, retrying (1/4) in 102 ms`, reprend son ajout sur l'état neuf, et valide. Les deux ajouts sont dans la table. Un ajout se rejoue sans risque ; une réécriture qui supprime des lignes, elle, doit vérifier que personne n'a touché aux mêmes fichiers.

© 2026 iAlexMG. Tous droits réservés.

Le catalogue est donc la seule pièce qui doit savoir remplacer une valeur de façon atomique. Au labo, c'est une base SQLite, `catalogue.db`, qui tient une ligne par table ; DuckDB 1.5 ne sait pas lui parler, et `lac_iceberg.sql` lit le chemin courant par l'extension `sqlite`. En entreprise, le catalogue est un service, que la spécification d'Iceberg décrit comme une API REST : Unity Catalog chez Databricks, Horizon et Open Catalog chez Snowflake, AWS Glue, Apache Polaris. Il ajoute ce qu'une base SQLite ne fait pas : les droits, l'audit, les noms partagés entre équipes. Delta, lui, peut se passer de catalogue : la dernière version du journal fait foi, et valider revient à créer le fichier de la version suivante à condition qu'il n'existe pas encore.

4. L'évolution du schéma : par identifiant, pas par nom

Labo : `Labo_10_DE.md`, étapes 3, 4 et 9.

Iceberg donne à chaque colonne un identifiant numérique, écrit dans le schéma de la table et dans chaque fichier Parquet qu'il produit. Le nom n'est qu'une étiquette sur l'identifiant. Au labo, le fichier de mars 2024 contient une colonne `temp_moteur_max` d'identifiant 9 ; le schéma 0 de la table appelle la colonne 9 `temp_moteur_max`, le schéma 1 l'appelle `temperature_moteur_max`, et crée la colonne 20, `heures_moteur`. Un lecteur qui demande la température cherche la colonne 9 dans chaque fichier, quel que soit le nom qu'elle y porte.

Changement	**Ce qu'il coûte dans Iceberg**	**Ce que voient les anciens fichiers**
Ajouter une colonne	un nouveau schéma, aucun fichier réécrit	la colonne est absente : `NULL`
Renommer	un nouveau schéma, aucun fichier réécrit	la même colonne, sous le nom neuf
Supprimer	un nouveau schéma, aucun fichier réécrit	la colonne est ignorée ; son identifiant n'est jamais réemployé
Élargir un type	permis pour `int` vers `long`, `float` vers `double`, une précision décimale plus grande	lue et convertie au vol
Rétrécir un type	refusé	rien ne change

La ligne de la suppression dit pourquoi l'identifiant d'une colonne supprimée ne revient jamais : une colonne neuve qui reprendrait le nom d'une ancienne ne doit pas en hériter les valeurs. Avec des noms, ce piège est inévitable ; avec des identifiants, il n'existe pas.

Delta repère ses colonnes par leur nom, par défaut. Renommer y veut dire réécrire la table, ou activer la correspondance des colonnes (*column mapping*), un mode qui donne à chaque colonne un identifiant et un nom physique, comme Iceberg. D'après la documentation de Databricks, consultée le 2026-09-23, ce mode se pose par la propriété `delta.columnMapping.mode`, `name` ou `id`, et permet alors `RENAME COLUMN` et `DROP COLUMN` sans réécriture ; il demande une version de protocole plus récente, que de vieux lecteurs ne savent pas lire, et un renommage peut casser une lecture en continu. Au labo, `deltalake` 1.6 refuse de l'activer. Sa seule évolution est la fusion de schéma, `schema_mode="merge"`, qui ajoute les colonnes inconnues : la table Delta finit avec les deux colonnes du bronze, 78 843 messages dans l'une et 125 642 dans l'autre.

© 2026 iAlexMG. Tous droits réservés.

Piège — un renommage qu'aucun outil ne voit. Le fichier Parquet de 2024 dit toujours `temp_moteur_max`. Un script qui le lit directement, par `read_parquet`, sans passer par la table, voit l'ancien nom et aucune colonne `heures_moteur`. Les fichiers d'une table ouverte ne se lisent que par la table : ses métadonnées sont la moitié de son contenu.

5. Le partitionnement caché

Labo : `Labo_10_DE.md`, étapes 4 et 8.

Le Cours 02 a partitionné le bronze par chemin, à la façon de Hive : la partition est un dossier, le dossier devient une colonne, et une requête ne saute des fichiers que si elle filtre sur cette colonne-là. Partitionner la télémétrie par mois demanderait une colonne `mois` que chaque écrivain calcule, que chaque lecteur connaît, et qu'aucun ne doit oublier ; changer de partition voudrait dire tout réécrire.

Iceberg partitionne par une transformation d'une colonne ordinaire. La table de la télémétrie déclare `MonthTransform` sur `jour` : la partition existe dans les métadonnées, pas dans les données, et la requête filtre sur `jour`, comme n'importe quelle requête. Au labo, un filtre sur octobre 2025 n'ouvre qu'un fichier sur 28.

Transformation	Ce qu'elle range ensemble
identité	les lignes d'une même valeur
année, mois, jour, heure	les lignes d'une même période, pour une date ou un horodatage
`bucket(N)`	les lignes dont la valeur tombe dans le même des N paniers, par hachage
`truncate(L)`	les lignes dont la valeur commence pareil, sur L caractères ou à L près

La partition peut aussi évoluer. Une nouvelle spécification vaut pour les données écrites ensuite ; les anciens fichiers gardent la leur, et le lecteur sait lire les deux. Un lac passé du jour au mois n'a rien à réécrire.

Piège — le mois se compte en UTC. Une transformation temporelle sur un horodatage avec fuseau travaille en temps universel. Les séries de l'usine s'arrêtent le 31 mars 2026 à 23 h, heure de l'Est ; leurs quatre dernières heures sont déjà le 1er avril à Greenwich, et la table de l'argent a une partition d'avril, avec 220 lignes. Rien n'est faux : une requête sur mars filtre sur l'heure et trouve ses lignes dans deux partitions. Mais une partition ne dit pas le mois local.

Delta partitionne par colonne, comme Hive : l'action `add` de chaque fichier porte la valeur de sa partition. Databricks recommande désormais le regroupement dynamique (*liquid clustering*) pour les tables neuves, qui laisse la plateforme ranger les lignes voisines ensemble sans partition fixe, comme le Cours 02 l'annonçait. Les deux formats convergent ainsi vers la même idée : la façon de ranger les fichiers appartient à la table, pas à la requête.

6. Les instantanés et le voyage dans le temps

Labo : `Labo_10_DE.md`, étapes 5 et 6.

© 2026 iAlexMG. Tous droits réservés.

Chaque validation d'une table Iceberg crée un instantané (*snapshot*) : l'état complet de la table à cet instant, désigné par un identifiant et décrit par sa liste de manifestes. Un instantané porte un résumé, où l'écrivain peut ajouter ses propres propriétés : au labo, chaque ajout y inscrit la période qu'il charge, `boreal.periode`, pour que l'historique se lise. Les instantanés se nomment par des références : la branche `main`, l'état courant, et des étiquettes (*tags*) posées à la main, comme `fin-2025` sur le rang 28.

Remonter le temps, c'est lire un instantané plutôt que le courant. Au labo, l'étiquette `fin-2025` rend 158 146 messages, et la comparaison au bronze arrêté au 31 décembre ne trouve rien en trop ni rien qui manque. Les usages sont concrets :

Besoin	**Ce que le voyage dans le temps donne**
Refaire un rapport	les chiffres du 31 décembre tels qu'on les a publiés, même si des corrections ont suivi
Auditer	ce que la table contenait quand une décision a été prise
Revenir en arrière	un chargement fautif se défait en republiant l'instantané d'avant
Déboguer	la différence entre deux états, ligne à ligne

Piège — l'heure des validations n'est pas celle des données. `snapshot_from_timestamp` cherche le dernier instantané validé avant l'heure donnée. Au labo, toute la table s'est écrite en une minute, en septembre 2026 ; demander l'état au 31 décembre 2025 par la date rend 0 message, et aucune erreur, puisqu'à cette heure-là la table n'existait pas. Une table porte deux horloges : celle des faits, dans ses colonnes (`jour`, `recu_le`), et celle de sa propre vie, dans ses instantanés. Le voyage dans le temps ne connaît que la seconde. Pour retrouver un état par la date des données, on étiquette l'instantané au moment où il compte, ou on consigne la période dans son résumé.

Delta numérote ses versions à partir de 0, et chaque version se relit, par son numéro ou par une heure de validation, avec la même réserve sur l'horloge. Delta n'a pas d'étiquette : pour garder une version au-delà de la rétention, Databricks la copie dans une autre table, par un clone profond (d'après sa documentation, consultée le 2026-09-23). Iceberg offre enfin des branches, des lignées d'instantanés parallèles à `main`, où l'on écrit et vérifie avant de publier.

7. L'entretien : compacter, expirer, nettoyer

Labo : `Labo_10_DE.md`, étapes 7 et 9.

Une table ouverte garde tout, et c'est ce qui la rend utile ; c'est aussi ce qui la fait grossir. Trois gestes la tiennent, dans cet ordre.

Compacter. Chaque ajout crée au moins un fichier : les 90 chargements quotidiens de 2026 laissent 90 petits fichiers, le piège des petits fichiers du Cours 02. Le compactage réécrit un mois en un fichier ; c'est une validation comme une autre, qui crée ses instantanés, et qui n'efface rien. Les lectures en cours continuent sur l'instantané qu'elles suivaient, dont les fichiers sont toujours là : c'est ce qui permet de compacter sans rien arrêter, promesse du Cours 02. Au labo, 90 fichiers deviennent 3, et une lecture de 2026 prend deux fois moins de temps, quelques centièmes de seconde de part et d'autre. Spark compacte en place, par la procédure `rewrite_data_files` ; PyIceberg 0.12 ne le sait pas, et le labo réécrit chaque mois par un `overwrite` limité au mois.

© 2026 iAlexMG. Tous droits réservés.

Expirer. Un instantané qu'on n'a plus besoin de relire s'oublie : il quitte les métadonnées, et le voyage dans le temps s'arrête là. D'après la documentation d'Iceberg, consultée le 2026-09-23, la procédure de Spark garde par défaut les instantanés de moins de cinq jours et au moins le dernier, épargne ceux qu'une étiquette ou une branche désigne, et efface les fichiers que seuls les instantanés expirés citaient. Au labo, 122 instantanés expirent ; l'étiquette `fin-2025` et l'état courant restent, et l'étiquette se relit toujours. Mais PyIceberg 0.12 n'efface aucun fichier en expirant : 121 fichiers de données restent sur le disque pour 31 cités.

Nettoyer. Un fichier que plus aucun instantané ne cite est orphelin : il occupe le stockage sans servir à personne. Le nettoyage les efface, avec une prudence qui compte. Un fichier récent peut appartenir à une écriture en cours, pas encore validée ; l'effacer ferait échouer sa validation. La procédure `remove_orphan_files` de Spark épargne donc par défaut tout ce qui a moins de trois jours, et le labo fait de même, 72 heures, sauf à le forcer. Forcé, il efface les 90 fichiers quotidiens.

Reste ce que le cours n'efface pas : les métadonnées. Chaque validation écrit un fichier `.metadata.json` qui recopie la liste des instantanés. À la fin du labo, les 126 fichiers de métadonnées pèsent 8,49 Mo, et les listes de manifestes et les manifestes 1,41 Mo de plus, pour 9,04 Mo de données. D'après la même documentation, deux propriétés de table bornent ce dossier : `write.metadata.delete-after-commit.enabled` efface les plus anciens à chaque validation, et `write.metadata.previous-versions-max`, 100 par défaut, dit combien en garder. Essayées à l'écriture de ce cours avec PyIceberg 0.12, et un maximum de 3, elles laissent 4 fichiers de métadonnées après 10 ajouts, contre 11 sans elles.

Delta suit le même ordre sous d'autres noms. `OPTIMIZE` compacte ; `VACUUM` efface les fichiers que les versions récentes ne citent plus, après une rétention de sept jours par défaut, que `delta.deletedFileRetentionDuration` règle. Le journal lui-même se garde trente jours par défaut. Au labo, le nettoyage forcé à zéro efface 31 fichiers, et la version 27 devient illisible : son journal existe encore, ses fichiers non.

Piège — effacer n'efface pas tant que l'historique garde. Un client qui exerce son droit à l'effacement, que la Loi 25 du Québec lui donne, fait retirer ses données de la table. Le voyage dans le temps les garde pourtant, dans les instantanés d'avant, tant qu'ils n'ont pas expiré ; et après l'expiration, dans les fichiers orphelins, tant qu'on ne les a pas nettoyés. Une suppression n'est complète qu'après les trois gestes, et une étiquette qui protège un vieil instantané protège aussi ce qu'il contient. Les droits des personnes appartiennent à la formation Architecture et Gouvernance ; ici, on voit seulement ce qu'un format de table y change.

8. Iceberg et Delta : deux formats, la même idée

Labo : `Labo_10_DE.md`, étape 9.

Les deux formats partent du même constat et arrivent au même résultat : des transactions, un schéma qui évolue, un historique, sur des fichiers Parquet. Ils diffèrent par l'histoire et par les détails, et les détails décident parfois d'un projet.

	Iceberg	**Delta Lake**
Né chez	Netflix, confié à la fondation Apache	Databricks, confié à la Linux Foundation
Métadonnées	un arbre, désigné par un catalogue	un journal, dans le dossier de la table

© 2026 iAlexMG. Tous droits réservés.

	Iceberg	Delta Lake
Colonnes	par identifiant, toujours	par nom, sauf avec la correspondance des colonnes
Partition	transformations cachées, qui peuvent évoluer	colonnes de partition, ou regroupement dynamique
Historique	instantanés, branches et étiquettes	versions numérotées
Entretien	réécriture, expiration, nettoyage des orphelins	`OPTIMIZE`, `VACUUM`
Au labo	PyIceberg 0.12 : renommage, partition cachée, étiquette	`deltalake` 1.6 : fusion de schéma seulement

La dernière ligne rappelle une leçon du Cours 07 : un format vaut ce que valent les outils qui l'écrivent là où vous travaillez. Le format Delta permet le renommage ; la bibliothèque du poste, pas encore. Choisir un format, c'est aussi choisir les moteurs qui le liront et l'écriront dans cinq ans.

La convergence est la tendance lourde, d'après les documentations consultées le 2026-09-23. Databricks crée des tables Iceberg gérées, et fait lire ses tables Delta par les clients Iceberg grâce à UniForm, qui écrit des métadonnées Iceberg à côté du journal. Snowflake lit des fichiers Delta à travers une table Iceberg. Microsoft Fabric présente chaque table Delta en Iceberg, et chaque table Iceberg en Delta. Le vrai choix se déplace ainsi du format vers le catalogue : c'est lui qui dit où sont les tables, qui y a droit, et quels moteurs les écrivent.

9. Le médaillon : bronze, argent, or

Labo : `Labo_10_DE.md`, étape 8.

L'architecture en médaillon range un lac en trois couches, du plus brut au plus prêt. Le nom vient des trois métaux, et chaque couche se désigne par le sien : bronze, argent (*silver*), or (*gold*).

Couche	Ce qu'elle garde	Qui peut la reconstruire
Bronze	la donnée telle qu'elle est arrivée, au plus près de la source, sans rien corriger	personne : c'est la copie de la source
Argent	la donnée nettoyée et rendue conforme : types posés, doublons retirés, unités et fuseaux ramenés, clés reconnues	le bronze, par le code qui la nettoie
Or	la donnée préparée pour un usage : agrégats, indicateurs, modèles en étoile	l'argent

© 2026 iAlexMG. Tous droits réservés.

Chez Boréal, le bronze tient la télémétrie et l'historien tels que livrés, en Parquet depuis le Cours 02 ; la télémétrie y est maintenant aussi en table Iceberg, aplatie mais pas corrigée. L'argent tient les trois séries de l'usine du Cours 08, nettoyées de leurs valeurs de départ, de leurs codes de qualité et de leur heure locale : le labo les met en tables Iceberg, identiques à celles de l'entrepôt ligne pour ligne. L'or n'est pas à Data Engineering : il appartient à ceux qui préparent la donnée pour un usage, et la section 10 dit lesquels.

Deux règles tiennent le médaillon. Chaque couche se reconstruit à partir de celle d'en dessous, et le bronze ne se reconstruit à partir de rien : c'est lui qu'on protège. Et une couche ne saute jamais la précédente, faute de quoi la correction faite en argent manque à l'or qui lit le bronze.

Une table ouverte a un prix de plus, celui de la traduction. Chaque moteur a ses types, et le format les ramène aux siens. Au labo, le `HUGEINT` de DuckDB, un entier de 128 bits qui n'existe pas dans Iceberg, sort en décimal de 38 chiffres : les valeurs sont les mêmes, un autre moteur verra un autre type. Publier une couche en table ouverte, c'est accepter que les types du format fassent foi.

Fabric. Microsoft Fabric fait du médaillon l'architecture recommandée de sa plateforme. Son lac, OneLake, est unique pour toute l'organisation et vient avec chaque locataire ; chaque couche y vit dans son propre lakehouse, ou dans un entrepôt pour l'or, et Microsoft conseille un espace de travail par couche. Tous les moteurs de Fabric écrivent en Delta par défaut, et OneLake présente chaque table Delta aussi en Iceberg, et une table Iceberg rattachée par un raccourci aussi en Delta, sous des limites que la documentation détaille : le renommage d'une colonne Iceberg, par exemple, ne passe pas en Delta. Pour l'argent et l'or, la documentation recommande le regroupement dynamique plutôt que le partitionnement. D'après la documentation de Microsoft, consultée le 2026-09-23.

10. Le médaillon et la chaîne `RAW`, `STAGING`, `MARTS`

Labo : `Labo_10_DE.md`, étape 8.

Le parcours emploie aussi une autre chaîne, celle d'Analytics Engineering : la `RAW` que Data Engineering livre depuis le Cours 03, puis `STAGING` et `MARTS`, que l'ingénieur analytique construit dans l'entrepôt. La ressemblance avec le médaillon est forte, et c'est ce qui rend la confusion coûteuse.

Médaillon	**Chaîne analytique**	**Où ça colle**	**Où ça cloche**
bronze	`RAW`	la donnée telle que livrée	le bronze garde des fichiers, la `RAW` des tables en texte, une par sorte de fichier livré
argent	`STAGING`	une donnée lisible, typée, nommée	l'argent joint volontiers plusieurs sources ; le `STAGING` s'y refuse, une vue par table source
or	`MARTS`	ce que les métiers consomment	l'or est souvent agrégé pour un usage ; les `MARTS` sont des étoiles au grain le plus fin, pas des agrégats

© 2026 iAlexMG. Tous droits réservés.

L'écart qui compte est au milieu, et le parcours y répond par un partage net. Data Engineering tient l'argent des séries de terrain, parce que les nettoyer demande de connaître la source : la bande morte d'un historien, ses codes de qualité, son archivage de minuit, l'heure locale d'un automate. C'est le schéma `argent` de l'entrepôt, que le labo publie aussi en Iceberg. Les sources relationnelles, elles, ne passent pas par un argent de Data Engineering : elles vont de la `RAW` au `STAGING` d'Analytics Engineering, qui connaît leur sens métier. Chacun nettoie ce qu'il comprend.

La comparaison complète, avec la chaîne analytique vue de son côté, est au Cours 01 de la formation Analytics Engineering ; ici, il suffit de savoir que le schéma `argent` porte bien de l'argent au sens du lakehouse, et pas un `STAGING`.

11. Le lakehouse, et la convergence du lac et de l'entrepôt

Le Cours 01 a opposé le lac, des fichiers bon marché dans n'importe quel format, et l'entrepôt, des tables typées avec leurs transactions et leurs droits. Longtemps, une entreprise a tenu les deux : un lac pour tout garder, un entrepôt pour servir, et des copies de l'un vers l'autre. Le lakehouse les fond. Des fichiers ouverts sur un stockage bon marché, un format de table pour les transactions et l'historique, un catalogue pour les noms et les droits, et plusieurs moteurs qui lisent la même copie.

	Lac	**Entrepôt**	**Lakehouse**
Stockage	fichiers ouverts, bon marché	format propre au moteur	fichiers ouverts, bon marché
Transactions	non	oui	oui, par le format de table
Schéma	décidé à la lecture	imposé à l'écriture	imposé à l'écriture, et qui évolue
Moteurs	tous ceux qui lisent les fichiers	le sien	tous ceux qui parlent le format et le catalogue
Ce qu'on paie	le désordre	la copie, et la dépendance au moteur	l'entretien des tables et du catalogue

La convergence va dans les deux sens. Les plateformes de lac ont ajouté le SQL, les droits et la gouvernance de l'entrepôt ; les entrepôts ont appris à lire et à écrire des tables ouvertes. Trois offres clés en main se disputent le terrain, et chacune a son encadré ci-dessous : Snowflake, Databricks et Fabric.

Pour Boréal, la question honnête est celle des besoins. Le lac tient sur un poste, et DuckDB lit ses tables sans serveur ; rien dans ce cours ne demande une plateforme. Ce que le format de table achète, c'est l'ouverture : la télémétrie en Iceberg se lit par DuckDB aujourd'hui, par Spark demain, par Snowflake ou Databricks si Boréal y va, sans copie ni export. Le prix, c'est l'entretien de la section 7, qui ne s'arrête jamais.

Snowflake. Snowflake tient des tables Iceberg de deux façons : Snowflake comme catalogue, avec tout ce qu'une table ordinaire sait faire, voyage dans le temps compris ; ou un catalogue externe, que Snowflake lit avec moins de fonctions. Depuis le 1er juin 2026, une table dont Snowflake est le catalogue peut laisser Snowflake ranger ses fichiers, sans stockage à configurer, dans les régions commerciales d'AWS et d'Azure ; les autres moteurs la lisent par le catalogue Horizon. Snowflake compacte lui-même ces tables quand il les écrit. D'après la documentation de Snowflake, consultée le 2026-09-23 ; l'annexe Snowflake du Labo 10 en montre les commandes.

© 2026 iAlexMG. Tous droits réservés.

Databricks. Une table créée sans autre précision y est une table Delta gérée par Unity Catalog ; `USING iceberg` en fait une table Iceberg gérée. Unity Catalog expose ses tables par le catalogue REST d'Iceberg, en lecture et en écriture, aux moteurs extérieurs. Sur les tables gérées, l'optimisation prédictive lance d'elle-même `OPTIMIZE`, `VACUUM` et `ANALYZE`, par défaut sur les comptes créés depuis le 11 novembre 2024. D'après la documentation de Databricks, consultée le 2026-09-23 ; l'annexe Databricks du Labo 10 refait l'évolution du schéma, renommage compris.

Fabric. Voir l'encadré de la section 9 : OneLake, un lac unique, des lakehouses par couche, Delta par défaut, et la présentation croisée en Iceberg.

Recoupements — ce que ce cours partage avec ses voisines.

- Analytics Engineering : bronze, argent et or recoupent `RAW`, `STAGING` et `MARTS` sans les recouvrir. Le parcours donne à Data Engineering l'argent des séries de terrain, et laisse les sources relationnelles aller de la `RAW` au `STAGING`. La chaîne analytique appartient à AE, qui en fait la comparaison à son Cours 01.
- Architecture et Gouvernance : le débat du lac et de l'entrepôt revient au Cours 01 d'Archi, et le choix de la pile, format et catalogue compris, au Cours 03. Ici, on en voit le mécanisme ; là-bas, on en décide.

Points clés à retenir

- Un format de fichier range des lignes ; un format de table dit quels fichiers forment la table.
- Iceberg décrit une table par un arbre de métadonnées, Delta par un journal ; les deux publient un nouvel état d'un seul geste.
- Valider, c'est remplacer un pointeur à condition qu'il n'ait pas bougé : la concurrence optimiste. Le catalogue tient ce pointeur.
- Un lecteur suit un état fixe ; pour voir la suite, il relit le pointeur.
- Iceberg désigne ses colonnes par identifiant : renommer, ajouter ou supprimer ne réécrit rien. Delta en fait autant avec la correspondance des colonnes, si l'outil sait l'activer.
- Le partitionnement caché dérive la partition d'une colonne ordinaire ; la requête filtre sur la colonne, et la table saute les fichiers. Le mois d'un horodatage se compte en UTC.
- Un instantané est un état complet ; une étiquette le garde. Le voyage dans le temps suit l'heure des validations, jamais celle des données.
- Compacter réunit sans effacer, expirer oublie des instantanés, nettoyer efface les orphelins, et les métadonnées se bornent par des propriétés de table.
- Une donnée effacée survit dans l'historique jusqu'à l'expiration et au nettoyage.
- Bronze garde, argent nettoie, or prépare. L'argent des séries de terrain est à Data Engineering ; les sources relationnelles vont de la `RAW` au `STAGING` d'Analytics Engineering.
- Le lakehouse, ce sont des fichiers ouverts, un format de table et un catalogue ; ce qu'il coûte, c'est leur entretien.

© 2026 iAlexMG. Tous droits réservés.

Exercices

Les énoncés et leur corrigé sont dans `Exercices_corriges_10_DE.md`. Faites le labo avant : plusieurs questions portent sur des chiffres que vous aurez mesurés vous-même.

© 2026 iAlexMG. Tous droits réservés.